

OT-Based SOC Incident Response Platform

Live Testing, Architecture, Production, and DevSecOps Guide

Repository: OT-Based-Soc-Incident-Response-Platform **Project type:** OT/ICS SOC incident-response MVP **Main goal:** convert real OPC UA simulator activity into analyst-readable SOC cases **Current lab split:** KEPServerEX on Windows, UaExpert and OPC UA monitor on Ubuntu **Document purpose:** explain how to test the live platform, how the project works from head to toe, how to move toward production, what is missing, and what hard problems were solved.

1. Executive Summary

This project is a human-supervised OT/ICS incident-response MVP. It uses a simulated water-management environment to prove that process activity, network evidence, and Wazuh alerts can be turned into a clear SOC investigation case. The project does not try to automatically shut down equipment or perform SOAR-style response. Its first mission is visibility, correlation, analyst review, and explainability.

The current live flow is:

```
UaExpert manual tag change
-> KEPServerEX OPC UA simulator
-> Python OPC UA monitor event
-> Wazuh process rule
-> Suricata network flow
-> Wazuh network rule
-> Django live ingestion
-> backend correlation
-> confirmed OPC UA case
-> Next.js SOC dashboard
```

The most important rule pair for a confirmed OPC UA operation is:

```
110203 or 110202 or 110204 etc. + 110104 = confirmed_opcua_operation
```

For the valve test:

```
110203 = VALF command tag changed
110104 = OPC UA network flow observed by Suricata
```

If the backend receives only one of these, it stores the live alert but does not create a confirmed case. A confirmed case needs both process evidence and network evidence.

2. Project Goal

The goal is to build a focused OT SOC incident-response platform for a simulated industrial control environment. The platform connects OT telemetry with SOC workflows. Instead of showing isolated raw alerts, it creates a case that tells the analyst:

- what tag changed
- which OPC UA node changed
- what the old and new values were
- which rule detected the process activity
- whether matching network evidence exists
- which host and file produced the evidence
- whether the case is validation, important, or suspicious

The current MVP is deliberately narrow. It focuses on OPC UA simulator traffic and Wazuh/Suricata evidence. This makes the project testable and explainable before adding larger features such as authentication, approval workflows, evidence hashing, risk scoring, or production SOC integrations.

3. Current Lab Architecture

3.1 Machines

The live lab is split across Windows and Ubuntu:

Windows host

- KEPServerEX simulator
- Suricata
- Windows Wazuh agent
- Docker backend/frontend/PostgreSQL stack, if running locally on Windows

Ubuntu VM

- UaExpert
- Python OPC UA monitor
- optionally Wazuh manager or Wazuh-side scripts, depending on deployment

Wazuh manager / indexer

- receives alerts from Ubuntu and Windows agents
- stores alerts.json and/or exposes indexer API

3.2 Network

The expected OPC UA endpoint is:

```
opc.tcp://<WINDOWS_IP>:49320
```

In the VirtualBox host-only lab, the common Windows IP is:

```
192.168.56.1
```

The Ubuntu VM must be able to reach KEPServerEX on port 49320. Suricata on Windows must capture the same network path so it can observe the OPC UA TCP flow.

3.3 Main Evidence Sources

The MVP uses two required evidence sources:

1. OPC UA process/tag evidence
2. Suricata network-flow evidence

Optional third evidence source:

3. KEPServerEX diagnostic evidence

The platform becomes strongest when all three exist, but a confirmed OPC UA case currently requires at least process evidence plus network evidence.

4. Major Components

4.1 KEPServerEX Simulator

KEPServerEX runs on Windows and exposes the OPC UA server. It simulates selected process tags such as:

```
VALF
MOTOR1
SU_SEVIYESI
```

It represents the OT process side of the project. UaExpert and the Python OPC UA monitor connect to it. In the current MVP, KEPServerEX is the source of real OPC UA state changes.

4.2 UaExpert

UaExpert runs on Ubuntu in the current lab. It is used as the manual operator/engineering client. During live testing, the user manually changes a tag value, for example:

```
VALF false -> true
```

This manual change is the starting point of the live test.

4.3 OPC UA

OPC UA is the industrial protocol being monitored. It carries the tag read/write/data-change behavior between UaExpert, Python monitor, and KEPServerEX. The MVP focuses on OPC UA because it gives a clear path from process value to network evidence to SOC case.

4.4 Python OPC UA Monitor

The Python monitor subscribes to selected OPC UA node IDs and writes JSON Lines events when values change. It does not control the process. It observes changes and writes process evidence.

Important file:

```
opcua-client/src/opcua_monitor.py
```

Example output file:

```
/home/ahmed_bahaa/ot-project/opcua-client/logs/opcua_monitor.jsonl
```

The Wazuh manager or agent reads this JSONL file and applies Wazuh OPC UA rules.

4.5 Suricata

Suricata runs on Windows and watches the adapter used for Ubuntu-to-Windows OPC UA communication. It writes network-flow evidence to eve.json. Wazuh reads this Suricata output and triggers the network-flow rule:

```
110104
```

This rule is essential because the backend uses it as network confirmation.

4.6 Wazuh Manager and Agents

Wazuh receives process and network evidence. In this lab:

- Ubuntu evidence can come from opcua_monitor.jsonl
- Windows evidence can come from Suricata eve.json
- Wazuh rules classify the events
- Wazuh manager stores alerts in alerts.json and/or Wazuh Indexer

The backend can ingest Wazuh alerts through:

1. File watcher: /var/ossec/logs/alerts/alerts.json
2. Wazuh/Indexer API polling
3. Vector HTTP forwarding

4.7 Wazuh Rules

Important rule files:

```
wazuh/rules/ot_opcua_rules.xml
wazuh/rules/zz_ot_simulator_mvp_rules.xml
```

Important rule IDs:

```
110103 = passive OPC UA datachange observed
110104 = OPC UA network flow observed by Suricata
110105 = KEPServerEX diagnostics write evidence
110200 = selected simulator tag changed
110201 = critical command tag changed
110202 = motor or pump command changed
110203 = valve command changed
110204 = water level tag changed
110205 = repeated failed OPC UA write attempts
```

4.8 Django Backend

The Django REST backend stores and exposes:

- cases
- evidence events
- live alerts
- Wazuh rule catalog
- OT tag catalog
- lab asset catalog

The backend now supports both:

```
manual case import
live Wazuh/Vector alert ingestion
```

Important endpoints:

```
GET /api/health/
GET /api/summary/
GET /api/cases/
GET /api/cases/{id}/
GET /api/evidence/
GET /api/live-alerts/
POST /api/ingest/wazuh-alerts/
POST /api/ingest/vector-alerts/
GET /api/docs/
```

4.9 PostgreSQL 18

SQLite is used for local development. PostgreSQL 18 is configured for Docker and production-style use. In Docker Compose, PostgreSQL runs as the db service.

Database URL:

```
postgresql://<POSTGRES_USER>:<POSTGRES_PASSWORD>@db:5432/<POSTGRES_DB>
```

For pgAdmin or host-side scripts on Windows, connect through the Docker-published port:

```
postgresql://<POSTGRES_USER>:<POSTGRES_PASSWORD>@127.0.0.1:5434/<POSTGRES_DB>
```

The Docker host port is intentionally 5434 to avoid colliding with a local PostgreSQL service already bound to 5433.

In production, credentials must be changed and stored as secrets, not committed to Git.

If an older local Docker volume was created with PostgreSQL 16, do not attach that same `/var/lib/postgresql/data` mount directly to PostgreSQL 18. PostgreSQL 18 Docker images expect the volume to be mounted at `/var/lib/postgresql`, with the actual database stored in a major-version-specific subdirectory. This project uses the `postgres18-data` volume and lets Django run migrations again. For

important data, perform a proper PostgreSQL dump/restore or major-version upgrade.

4.10 Next.js Frontend

The frontend is a SOC-style dashboard. It reads the backend REST API and displays:

- dashboard summary
- confirmed OPC UA cases
- evidence timelines
- Wazuh rules
- OT tags
- lab assets

The frontend refreshes open pages every five seconds, so newly created backend cases can appear without manual browser refresh.

4.11 Docker Compose

Docker Compose runs:

```
db          = PostgreSQL
backend     = Django REST API
frontend    = Next.js dashboard
```

The frontend uses two API URLs:

```
OT_SOC_API_BASE_URL=http://backend:8000/api
NEXT_PUBLIC_OT_SOC_API_BASE_URL=http://127.0.0.1:8000/api
```

The first is used inside Docker. The second is used by the browser for visible API links.

5. How the Project Works From Head to Toe

5.1 Live Case Creation Flow

The live case begins with a manual change in UaExpert:

```
VALF false -> true
```

The Python OPC UA monitor receives a subscription notification and writes a JSON event:

```
event.action = opcua_datachange
ot.tag = VALF
ot.old_value = false
ot.new_value = true
```

Wazuh reads this event and applies simulator rules. For VALF, Wazuh should produce:

```
110203
```

At the same time, Suricata on Windows observes OPC UA traffic to the KEPServerEX port:

```
destination port = 49320
protocol = TCP
event_type = flow
```

Wazuh reads Suricata eve.json and applies:

```
110104
```

The backend live ingestion service receives these alerts and stores them as LiveAlert rows. It then checks a time window, usually 900 seconds, and tries to pair process evidence with network evidence.

When the backend sees:

```
110203 + 110104
```

it creates:

```
case_type = confirmed_opcua_operation
classification = suspicious_ot_operation
tag = VALF
rule_ids = ["110104", "110203"]
```

The frontend dashboard reads /api/cases/ and displays the case.

5.2 Why Two Evidence Sources Are Required

The platform intentionally avoids creating a confirmed case from one isolated alert. If only the Python monitor sees a data change, the event may be incomplete. If only Suricata sees a flow, it does not know which tag changed.

The project becomes more reliable when it can say:

```
The process tag changed, and matching OPC UA network traffic was observed.
```

This is why the backend correlates process and network evidence before creating the case.

6. Exact Live Testing Procedure

This is the procedure to test the real platform using UaExpert manual changes.

6.1 Step 1: Start KEPServerEX on Windows

On Windows, start KEPServerEX and make sure the OPC UA endpoint is reachable:

```
opc.tcp://<WINDOWS_IP>:49320
```

Common lab example:

```
opc.tcp://192.168.56.1:49320
```

Make sure KEPServerEX certificates/trust are already configured so UaExpert and Python clients can connect.

6.2 Step 2: Start Docker Backend and Frontend

On Windows, from the repository root:

```
docker compose up --build
```

Check backend:

```
Invoke-RestMethod http://127.0.0.1:8000/api/health/
```

Expected:

```
{
  "status": "ok",
  "service": "ot-soc-backend"
}
```

Open:

```
http://127.0.0.1:3000
http://127.0.0.1:8000/api/docs/
```

6.3 Step 3: Start Suricata on Windows

Suricata must capture the network adapter used between Ubuntu and Windows. It should write `eve.json`.

Expected output path in the current lab:

```
C:\OT-Project\suricata-output\flow-proof-current\eve.json
```

Wazuh Windows agent must collect this file and send it to the Wazuh manager.

Expected Wazuh rule:

```
110104
```

6.4 Step 4: Start the Python OPC UA Monitor on Ubuntu

On Ubuntu:

```
cd /home/ahmed_bahaa/ot-project/opcua-client
source .venv/bin/activate
```

Do not run the monitor with `sudo`. The virtual environment contains the `asyncua` package, and `sudo python3` bypasses that environment.

If KEPServerEX rejects anonymous OPC UA sessions, set the same credentials that work from UaExpert:

```
export OPCUA_ENDPOINT="opc.tcp://192.168.56.1:49320"
export OPCUA_USERNAME="<KEPSERVER_USERNAME>"
export OPCUA_PASSWORD="<KEPSERVER_PASSWORD>"
```

Start monitor for VALF:

```
python src/opcua_monitor.py \
  --node-id "ns=2;s=watersim.TankPLC.MAIN.ARTIRMA_VERI.VALF" \
  --scenario-id "uaexpert-live-test" \
  --interval-ms 1000
```

Expected message:

```
[+] Monitor is running.  
[+] Now change the tag from UaExpert GUI.
```

The monitor writes:

```
/home/ahmed_bahaa/ot-project/opcua-client/logs/opcua_monitor.jsonl
```

Wazuh should later detect:

```
110203
```

6.5 Step 5: Start Backend Live Ingestion

You need one of the following live ingestion methods.

Option A: Poll Wazuh Indexer from the Backend Container

Use this if Docker backend on Windows can reach Wazuh Indexer:

```
Copy-Item .env.example .env  
# Edit .env:  
# WAZUH_INDEXER_ALERTS_URL=https://<WAZUH_INDEXER_IP>:9200/wazuh-alerts-*/_search  
# WAZUH_API_USERNAME=<INDEXER_USER>  
# WAZUH_API_PASSWORD=<INDEXER_PASSWORD>  
# WAZUH_API_INSECURE=true  
  
docker compose --profile wazuh-poller up -d wazuh-poller
```

For a one-time poll test from the already running backend container:

```
docker compose exec backend python manage.py poll_wazuh_alerts --once
```

In production, avoid `--insecure`. Use valid TLS certificates.

The 9200 endpoint uses Wazuh Indexer/OpenSearch credentials. These are not necessarily the same as Wazuh server API credentials on 55000.

If the Wazuh installation assistant was used, print the generated passwords on the Ubuntu Wazuh node:

```
sudo tar -O -xvf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt
```

To show only the admin indexer user:

```
sudo tar -axf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt -O | grep -P "'admin'" -A 1
```

If the archive is gone, reset the indexer password on a Wazuh indexer node:

```
sudo /usr/share/wazuh-indexer/plugins/opensearch-security/tools/wazuh-passwords-tool.sh \  
-u admin \  
-p '<NEW_INDEXER_PASSWORD>'
```

Option B: Watch Wazuh alerts.json

Use this if backend code is running on the Wazuh manager:

```
python manage.py watch_wazuh_alerts \  
--file /var/ossec/logs/alerts/alerts.json \  
--wait \  
--window-seconds 900
```

Option C: Use Vector to POST to Django

Use the provided Vector configuration:

```
vector/django-live-ingest.toml
```

Vector should post to:

```
http://<BACKEND_IP>:8000/api/ingest/vector-alerts/
```

For example, if the backend is on the Windows host:

```
http://192.168.56.1:8000/api/ingest/vector-alerts/
```

6.6 Step 6: Change the Tag in UaExpert

On Ubuntu, open UaExpert and connect to:

```
opc.tcp://<WINDOWS_IP>:49320
```

Find:

```
VALF
```

Change:

```
false -> true
```

6.7 Step 7: Confirm Wazuh Alerts

On Wazuh manager:

```
sudo tail -f /var/ossec/logs/alerts/alerts.json | grep -E "110203|110104"
```

You need to see both:

```
110203 = valve command changed  
110104 = OPC UA network flow observed
```

6.8 Step 8: Confirm Backend Live Alerts

On Windows:

```
Invoke-RestMethod http://127.0.0.1:8000/api/live-alerts/
```

Expected:

```
rule_id: 110203  
rule_id: 110104
```

6.9 Step 9: Confirm Backend Case

```
Invoke-RestMethod http://127.0.0.1:8000/api/cases/
```

Expected:

```
case_type: confirmed_opcua_operation  
tag: VALF  
classification: suspicious_ot_operation  
rule_ids: 110104, 110203
```

6.10 Step 10: Confirm Dashboard

Open:

```
http://127.0.0.1:3000/cases
```

The case should appear automatically within about five seconds.

7. Debugging Checklist

If the dashboard does not show the case, debug in this order.

7.1 Did UaExpert Connect?

Check:

```
UaExpert connected to opc.tcp://<WINDOWS_IP>:49320
```

If not:

- check KEPServerEX running
- check Windows firewall
- check OPC UA endpoint
- check certificates/trust
- check VirtualBox host-only network

7.2 Did the Python Monitor Print the Change?

The monitor should print a JSON event when the tag changes. If not:

- check the node ID
- check the monitor certificate
- check KEPServerEX trust
- check that the monitor is subscribed before changing the tag

7.3 Did Wazuh Produce Rule 110203?

On Wazuh manager:

```
sudo tail -f /var/ossec/logs/alerts/alerts.json | grep 110203
```

If missing:

- Wazuh may not be reading `opcua_monitor.jsonl`
- rule file may not be deployed
- Wazuh manager may need restart
- JSON format may not match the decoder/rule

7.4 Did Suricata Produce Network Evidence?

Check Suricata `eve.json` on Windows. It should contain OPC UA TCP flow to port 49320.

If missing:

- Suricata may be watching the wrong adapter
- the flow may be on another interface
- Windows firewall may affect visibility
- Suricata may not be running

7.5 Did Wazuh Produce Rule 110104?

On Wazuh manager:

```
sudo tail -f /var/ossec/logs/alerts/alerts.json | grep 110104
```

If missing:

- Windows Wazuh agent may not collect Suricata `eve.json`

- Suricata path may differ
- Wazuh rule 110104 may not be deployed
- source/destination fields may not match expected rule fields

7.6 Did Backend Receive the Alerts?

Invoke-RestMethod http://127.0.0.1:8000/api/live-alerts/

If missing:

- Wazuh polling command may not be running
- Vector may not be posting
- backend container may not reach Wazuh API
- backend port may not be reachable from Wazuh/Vector

7.7 Did Backend Create the Case?

Invoke-RestMethod http://127.0.0.1:8000/api/cases/

If alerts exist but case does not:

- maybe only one rule arrived
- timestamps may be outside the correlation window
- flow alert may not include destination IP/port
- process alert may not include tag/node ID

7.8 Did Frontend Refresh?

Open:

http://127.0.0.1:3000/cases

If backend has the case but frontend does not:

- check frontend container is running
- check OT_SOC_API_BASE_URL
- check NEXT_PUBLIC_OT_SOC_API_BASE_URL
- check browser console
- reload the page once

8. Database Checks

8.1 PostgreSQL in Docker

List latest cases:

```
docker compose exec db psql -U ot_soc -d ot_soc -c "select id, tag, classification, created_at_from_case from soc_case order by created_at desc limit 10;"
```

List live alerts:

```
docker compose exec db psql -U ot_soc -d ot_soc -c "select id, source, rule_id, timestamp from soc_livealert order by timestamp desc limit 10;"
```

Count cases:

```
docker compose exec db psql -U ot_soc -d ot_soc -c "select count(*) from soc_case;"
```

8.2 Django Shell

Inside backend container:

```
docker compose exec backend python manage.py shell
```

Then:

```
from soc.models import Case, LiveAlert, EvidenceEvent
print(Case.objects.count())
print(LiveAlert.objects.count())
print(EvidenceEvent.objects.count())
```

9. Manual API Test Without Wazuh

Before testing the real lab, you can test backend live ingestion using two sample alerts:

```
$alerts = @(
    @{
        timestamp="2026-07-12T12:00:00.000000Z"
        rule=@{id="110203"; description="OT simulator valve command changed"}
        agent=@{name="Ubuntu"}
        location="opcua_monitor.jsonl"
        data=@{ot=@{tag="VALF"; node_id="ns=2;s=watersim.TankPLC.MAIN.ARTIRMA_VERI.VALF"; old_value=$false; new_value=$true}}
    },
    @{
        timestamp="2026-07-12T12:00:00.250000Z"
        rule=@{id="110104"; description="OT OPC UA network flow observed by Suricata"}
        agent=@{name="My-Win-Machine"}
        location="eve.json"
        data=@{src_ip="192.168.56.10"; dest_ip="192.168.56.1"; dest_port="49320"}
    }
)

Invoke-RestMethod `
    -Method Post `
    -Uri "http://127.0.0.1:8000/api/ingest/wazuh-alerts/?window_seconds=60" `
    -ContentType application/json `
    -Body ($alerts | ConvertTo-Json -Depth 10)
```

Expected:

```
alerts_created = 2
cases_created = 1
evidence_created = 2
```

This proves backend correlation and frontend display before debugging the real OT lab.

10. DevSecOps Lifecycle Implemented So Far

The project now has a practical DevSecOps loop:

```
code change
-> tests
-> validation
-> security scans
-> Docker validation
-> review
-> lab deployment
```

Implemented controls:

- Django tests
- Django migration drift check
- repository validation tests
- OPC UA client ruff and pytest
- frontend lint and build
- OpenAPI schema validation
- Docker Compose validation
- dependency audits
- Bandit static analysis
- Gitleaks secret scanning
- Dependabot configuration

Pipeline files:

```
.github/workflows/ci.yml
.github/workflows/security.yml
.github/dependabot.yml
.gitleaks.toml
scripts/validate-ci.sh
scripts/validate-local.ps1
```

Local validation:

```
.\scripts\validate-local.ps1
```

CI validation:

```
./scripts/validate-ci.sh
```


11. Production Deployment Goal

The production goal is not simply "run Docker on a server." Production means the platform is:

- reachable only through controlled entry points
- protected with TLS
- configured with secrets
- backed by PostgreSQL
- monitored
- logged
- recoverable
- deployed through CI/CD
- separated from direct OT control actions

The project should remain human-reviewed. It should not automatically shut down equipment or modify process values.

12. Recommended Production Architecture

12.1 Minimum Production-Like Architecture

- Cloud VM or on-prem VM
- Docker Engine
- PostgreSQL container or managed PostgreSQL
- Django backend container
- Next.js frontend container
- NGINX reverse proxy
- TLS certificates
- SSH access for admins only

- Wazuh manager / indexer
- sends alerts to backend
- or backend polls Wazuh Indexer

- OT lab network
- KEPServerEX
- Suricata
- Wazuh agents
- OPC UA monitor

12.2 Network Segmentation

Recommended network exposure:

- Public Internet:
 - 443 only, through NGINX
- Admin:
 - SSH 22 only from trusted IPs or VPN
- Internal:
 - backend 8000 not public
 - frontend 3000 not public
 - PostgreSQL 5432/5433/5434 not public
 - Wazuh API allowed only from backend or trusted network

PostgreSQL should not be exposed to the internet.

12.3 Reverse Proxy

NGINX should terminate TLS and proxy traffic to frontend/backend containers.

Example structure:

- `https://soc.example.com/` -> frontend
- `https://soc.example.com/api/` -> backend
- `https://soc.example.com/admin/` -> backend admin, preferably restricted

NGINX official documentation describes reverse proxying using `proxy_pass`, and HTTPS configuration with TLS settings. Django official deployment documentation also recommends reviewing settings carefully before exposing a Django project to the internet.

References:

- Django deployment checklist: <https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/>
- Django security overview: <https://docs.djangoproject.com/en/6.0/topics/security/>
- NGINX proxy module: https://nginx.org/en/docs/http/nginx_http_proxy_module.html
- NGINX HTTPS configuration: https://nginx.org/en/docs/http/configuring_https_servers.html

13. Example Production NGINX Configuration

This is a production-style starting point, not a final hardened config:

```
server {
    listen 80;
    server_name soc.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name soc.example.com;

    ssl_certificate /etc/letsencrypt/live/soc.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/soc.example.com/privkey.pem;

    client_max_body_size 10m;

    location /api/ {
        proxy_pass http://127.0.0.1:8000/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }

    location /admin/ {
        allow <TRUSTED_ADMIN_IP>;
        deny all;

        proxy_pass http://127.0.0.1:8000/admin/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

Production note:

- Do not expose Django runserver directly.
- Do not expose PostgreSQL directly.
- Do not expose Wazuh credentials in files committed to Git.

14. Production Django Settings

For production, set:

```
DJANGO_DEBUG=false
DJANGO_SECRET_KEY=<strong secret>
DJANGO_ALLOWED_HOSTS=soc.example.com
DJANGO_CORS_ALLOWED_ORIGINS=https://soc.example.com
DJANGO_CSRF_TRUSTED_ORIGINS=https://soc.example.com
DJANGO_SECURE_SSL_REDIRECT=true
DJANGO_SESSION_COOKIE_SECURE=true
DJANGO_CSRF_COOKIE_SECURE=true
DJANGO_SECURE_HSTS_SECONDS=31536000
DJANGO_SECURE_HSTS_INCLUDE_SUBDOMAINS=true
DJANGO_SECURE_HSTS_PRELOAD=true
DATABASE_URL=postgres://user:password@db-host:5432/ot_soc
```

Django's deployment checklist should be reviewed before production. It specifically highlights settings, security, performance, and operational readiness.

Reference:

<https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/>

15. SSH and VM Hardening

For a cloud or on-prem VM:

1. Use SSH keys, not passwords.
2. Disable root login.
3. Restrict SSH to trusted IPs or VPN.
4. Keep the OS patched.
5. Run Docker as a controlled service.
6. Use a firewall.
7. Keep only 80/443 public.
8. Store secrets outside Git.
9. Back up PostgreSQL.
10. Monitor disk space and logs.

Example firewall intent:

```
allow 22/tcp from admin IP only
allow 80/tcp public
allow 443/tcp public
deny 5432/tcp public
deny 5433/tcp public
deny 5434/tcp public
deny 8000/tcp public
deny 3000/tcp public
```

If possible, use a VPN or private network for administrative access.

16. Production Docker Strategy

16.1 Current Docker Compose

The current Compose file is good for lab and production-like testing. It runs:

```
db
backend
frontend
```

It also uses health checks and dependency ordering. Docker Compose official documentation describes service definitions, health checks, and startup order.

References:

- Compose file reference: <https://docs.docker.com/reference/compose-file/>
- Compose services reference: <https://docs.docker.com/reference/compose-file/services/>
- Compose startup order: <https://docs.docker.com/compose/how-tos/startup-order/>

16.2 Production Improvements

Before production:

- use non-default PostgreSQL credentials
- use Docker secrets or cloud secrets
- do not mount sensitive host paths unnecessarily
- store database backups outside the VM
- add restart policies
- add log rotation
- add image vulnerability scanning
- pin image versions
- separate staging and production environments

Docker secrets are designed to manage sensitive data so it is not stored directly in application source code or Dockerfiles.

Reference:

<https://docs.docker.com/engine/swarm/secrets/>

17. CI/CD Production Pipeline

17.1 Current Pipeline

The current CI pipeline should run on pull requests and pushes:

```
backend tests
repository validation
OPC UA client tests
frontend lint/build
OpenAPI validation
Docker Compose validation
security scans
dependency audits
secret scanning
```

17.2 Recommended Production Pipeline

For production, split CI/CD into stages:

1. Pull request validation
2. Security scans
3. Docker image build
4. Image vulnerability scan
5. Push image to registry
6. Deploy to staging
7. Smoke test staging
8. Manual approval
9. Deploy to production
10. Smoke test production
11. Rollback if health checks fail

17.3 Deployment by SSH

Simple production deployment can be:

```
GitHub Actions -> SSH to VM -> docker compose pull -> docker compose up -d
```

But this requires SSH private key handling. Use GitHub secrets carefully and restrict the deployment key.

17.4 Deployment by OIDC

GitHub Actions supports OpenID Connect so workflows can authenticate to cloud providers without long-lived cloud secrets. This is better for production cloud deployments.

References:

- GitHub Actions security: <https://docs.github.com/actions/security-for-github-actions>
- GitHub secure use reference: <https://docs.github.com/en/actions/reference/security/secure-use>
- GitHub OIDC: <https://docs.github.com/en/actions/concepts/security/openid-connect>

18. Example CI/CD Deployment Flow

Example high-level workflow:

```
Developer opens PR
-> CI runs tests, lint, audits, Docker build
-> reviewer approves
-> merge to main
-> build backend image
-> build frontend image
-> push images to registry
-> SSH or OIDC deploy to staging
-> run smoke tests
-> manual approval
-> deploy to production
```

Production smoke tests:

```
GET /api/health/
GET /api/docs/
GET /api/cases/
GET frontend /
POST safe test alert to staging only
```

Do not run destructive tests against live production OT systems.

19. Production Wazuh Integration

There are three main options.

19.1 Poll Wazuh Indexer

Backend polls:

```
https://<WAZUH_INDEXER>:9200/wazuh-alerts-*/_search
```

Pros:

- simple to control from backend
- no need to install Vector
- easy to test

Cons:

- requires credentials
- polling interval adds delay
- backend needs network access to Wazuh Indexer

19.2 Vector Push

Vector tails Wazuh alerts and pushes matching OT rules to Django.

Pros:

- near-real-time
- backend does not need Wazuh credentials
- good event-forwarding pattern

Cons:

- Vector must be installed and managed
- backend endpoint must be reachable from Vector
- authentication should be added before internet exposure

19.3 File Watcher

Backend watches:

```
/var/ossec/logs/alerts/alerts.json
```

Pros:

- simple
- no Wazuh API credentials

Cons:

- only works if backend runs on Wazuh manager or has safe file access
- less clean for distributed production

Recommendation:

Lab: file watcher or API poller

Production-like staging: Wazuh Indexer polling

Future production: Vector push or authenticated ingestion endpoint

20. What Is Missing

The project is a strong MVP, but not a full enterprise SOC platform yet.

Missing product features:

- user authentication
- analyst roles
- case status lifecycle
- analyst notes
- comments
- evidence hashing
- chain-of-custody timeline
- alert suppression rules
- advanced risk scoring
- asset ownership context
- maintenance window context
- acknowledgement workflow
- final report export

Missing production features:

- hardened NGINX deployment
- TLS certificates
- domain name
- cloud/on-prem VM automation
- image registry
- production CI/CD deployment stage
- image vulnerability scanning
- centralized logging
- monitoring and alerting
- database backups
- restore testing
- authentication for ingest endpoints
- rate limiting
- network allowlisting

Missing OT coverage:

- broader tag catalog
- more protocols such as Modbus, S7, DNP3, or EtherCAT
- real PLC/RTU behavior
- real physical process validation
- richer KEPServerEX diagnostics
- more Suricata protocol rules

21. What Could Be Better

21.1 Backend

The backend can improve by adding:

- authenticated ingestion
- case status fields
- analyst notes
- evidence hash fields
- audit log model
- risk scoring model
- async ingestion workers
- retention policy for live alerts
- PostgreSQL indexes based on real query performance

21.2 Frontend

The frontend can improve by adding:

- live WebSocket updates instead of polling
- case status controls
- timeline visualization
- filtering by tag, rule, source, and time
- evidence diff view
- exportable report view
- better mobile layout
- role-based views after authentication exists

21.3 DevSecOps

DevSecOps can improve by adding:

- image vulnerability scanning
- SBOM generation
- artifact signing
- staging deployment job
- production manual approval
- rollback automation
- cloud OIDC deployment
- backup restore validation

21.4 OT Detection

Detection can improve by adding:

- rule tuning for normal engineering changes
- maintenance window awareness

- asset criticality scoring
- multiple tag sequence correlation
- repeated write pattern detection
- command source attribution
- physical process anomaly context

22. Hardest Problems Overcome

22.1 Connecting OT Evidence to SOC Cases

The hardest conceptual problem was moving from isolated telemetry to an analyst-readable case. A single log line is not enough for SOC work. The project solved this by combining process evidence and network evidence into one `confirmed_opcua_operation` case.

22.2 Making the MVP Real Without Overbuilding

It would have been easy to design a huge SOC platform. Instead, the project stayed focused on one validated workflow:

```
OPC UA tag change -> Wazuh/Suricata evidence -> correlated case -> dashboard
```

This focus made the MVP testable.

22.3 Handling Windows and Ubuntu Split

The real lab is split across Windows and Ubuntu. KEPServerEX and Suricata are on Windows, while UaExpert and the Python monitor are on Ubuntu. This creates network, certificate, file path, and telemetry routing challenges.

The project overcame this by clearly separating:

- process evidence from Ubuntu
- network evidence from Windows
- Wazuh correlation at the backend

22.4 Avoiding Manual Correlation

Originally, the correlation script and import command were manual. The platform now supports live ingestion. This means the backend can receive Wazuh/Vector alerts, store them, correlate them, and create cases automatically.

22.5 Making the Frontend Backend-Ready

The frontend was adjusted to consume the real backend API instead of static demo data. It now reads cases, evidence, rules, tags, and assets from Django.

22.6 Building DevSecOps Around an OT Project

DevSecOps for OT is not only normal web testing. The project also validates Wazuh rules, Vector config, Docker Compose, and correlation fixtures. This protects the detection engineering contract, not only the application code.

23. Final Demonstration Script

Use this script when demonstrating the project.

23.1 Introduction

Say:

This project is an OT SOC incident-response MVP. I will manually change an OPC UA simulator tag using UaExpert, then

23.2 Show Running Services

Show:

```
docker compose ps
Invoke-RestMethod http://127.0.0.1:8000/api/health/
```

Open:

```
http://127.0.0.1:3000
```

23.3 Show Monitor

On Ubuntu:

```
python src/opcua_monitor.py \
--node-id "ns=2;s=watersim.TankPLC.MAIN.ARTIRMA_VERI.VALF" \
--scenario-id "demo-live-test" \
--interval-ms 1000
```

23.4 Show Wazuh Live Ingestion

Run one ingestion method:

```
docker compose --profile wazuh-poller up -d wazuh-poller
```

23.5 Perform UaExpert Change

In UaExpert:

```
VALF false -> true
```

23.6 Show Evidence

Show:

```
Invoke-RestMethod http://127.0.0.1:8000/api/live-alerts/
Invoke-RestMethod http://127.0.0.1:8000/api/cases/
```

Then show:

```
http://127.0.0.1:3000/cases
```

23.7 Explain Result

Say:

The dashboard case was created because the backend received both process evidence and network-flow evidence. This pro

24. Quick Command Appendix

Start Docker

```
docker compose up --build
```

Backend Health

```
Invoke-RestMethod http://127.0.0.1:8000/api/health/
```

Swagger

```
http://127.0.0.1:8000/api/docs/
```

Frontend

```
http://127.0.0.1:3000
```

Start OPC UA Monitor

```
python src/opcua_monitor.py \  
  --node-id "ns=2;s=watersim.TankPLC.MAIN.ARTIRMA_VERI.VALF" \  
  --scenario-id "uaexpert-live-test" \  
  --interval-ms 1000
```

Poll Wazuh Indexer

```
docker compose exec backend python manage.py poll_wazuh_alerts --once
```

Watch Wazuh Alerts File

```
python manage.py watch_wazuh_alerts \  
  --file /var/ossec/logs/alerts/alerts.json \  
  --wait \  
  --window-seconds 900
```

Check Live Alerts

```
Invoke-RestMethod http://127.0.0.1:8000/api/live-alerts/
```

Check Cases

```
Invoke-RestMethod http://127.0.0.1:8000/api/cases/
```

Check PostgreSQL Cases

```
docker compose exec db psql -U ot_soc -d ot_soc -c "select id, tag, classification from soc_case order by id desc limit 10"
```

Local Validation

```
.\scripts\validate-local.ps1
```


25. References

These references were used for production and DevSecOps guidance:

- Django deployment checklist: <https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/>
- Django security overview: <https://docs.djangoproject.com/en/6.0/topics/security/>
- Django static files deployment: <https://docs.djangoproject.com/en/6.0/howto/static-files/deployment/>
- Docker Compose file reference: <https://docs.docker.com/reference/compose-file/>
- Docker Compose services reference: <https://docs.docker.com/reference/compose-file/services/>
- Docker Compose startup order and health checks: <https://docs.docker.com/compose/how-tos/startup-order/>
- Docker secrets: <https://docs.docker.com/engine/swarm/secrets/>
- GitHub Actions security: <https://docs.github.com/actions/security-for-github-actions>
- GitHub Actions secure use reference: <https://docs.github.com/en/actions/reference/security/secure-use>
- GitHub Actions OpenID Connect: <https://docs.github.com/en/actions/concepts/security/openid-connect>
- NGINX proxy module: https://nginx.org/en/docs/http/nginx_http_proxy_module.html
- NGINX HTTPS configuration: https://nginx.org/en/docs/http/configuring_https_servers.html

26. Final Notes

This project has reached a meaningful MVP stage. It can show a real chain from manual OPC UA change to dashboard case, as long as the Wazuh and Suricata evidence path is correctly connected.

The best next phase is not to add many features at once. The best next phase is:

1. make the live lab test repeatable
2. add analyst case lifecycle
3. add evidence hashing and audit timeline
4. harden production deployment
5. expand OT detection coverage

The most important engineering principle for the project is:

Do not automate control response before visibility, evidence quality, and human approval are trustworthy.